

Client-Server Socket Programming Report

Haneen Hamed Mohamed	
Seif eldin Mansour Mohamed	
Malak Adel Okda	
Mennatallah Elsayed Mohamed	

Title

Multi-Threaded Client-Server Communication System Using Python Sockets

Objective

The objective of this project is to design and implement a TCP-based client-server system using Python. The system enables multiple clients to connect to a server concurrently, send structured commands, and receive processed responses. The server also logs all activities in a thread-safe manner.

Technologies Used

- Python Programming Language
 - `socket` library for network communication
 - `threading` library for handling multiple clients
 - `datetime` module for time/date operations and logging
-

System Overview

Server

The server is responsible for:

- Listening for incoming client connections
- Handling multiple clients simultaneously using threads
- Processing commands received from clients
- Sending results back to clients

- Logging all interactions into a file

Client

The client is responsible for:

- Connecting to the server
 - Sending user-entered commands
 - Receiving responses from the server
 - Displaying results to the user
-

Communication Protocol

The communication between client and server follows a structured format:

COMMAND:MESSAGE

Example:
UPPER:hello

Response:
HELLO

If the message format is invalid (missing ':'), the server returns an error.

Server Implementation

Server Configuration

The server runs on:

- IP Address: 127.0.0.1 (localhost)
- Port: 4949

Socket Creation

A TCP socket is created using:

- IPv4 addressing (`AF_INET`)
- Stream-based communication (`SOCK_STREAM`)

Binding and Listening

The server binds to the specified IP and port, then starts listening for incoming connections.

Accepting Clients

The server continuously accepts new client connections and assigns each client a unique name (Client1, Client2, etc.).

Multi-Threading

Each client is handled in a separate thread. This allows multiple clients to interact with the server simultaneously without blocking each other.

Client Handling

For each connected client:

1. The server receives data from the client
2. Validates the message format
3. Processes the command
4. Sends the result back
5. Logs the interaction

Command Processing

The server supports the following commands:

- UPPER: Converts text to uppercase
- LOWER: Converts text to lowercase
- REVERSE: Reverses the string
- COUNT: Returns the number of characters
- VOWELS: Counts the number of vowels
- TIME: Returns current server time
- DATE: Returns current server date

If an unknown command is received, an error message is returned.

Thread Safety

Two locks are implemented to ensure safe concurrent operations:

counter_lock

Ensures that each client receives a unique identifier without race conditions.

log_lock

Ensures that multiple threads do not write to the log file simultaneously, preventing data corruption.

Logging System

All server activities are recorded in a file named `server_log.txt`. Each log entry includes:

- Timestamp
- Client identifier
- Command received
- Result sent

Example log entry:

2026-04-11 15:00:00: Client1 sent: UPPER: hello -> HELLO

Client Implementation

Connection

The client connects to the server using the same IP address and port.

User Input

The client continuously prompts the user to enter commands.

Data Transmission

- User input is encoded and sent to the server
- The client waits for a response from the server
- The received response is decoded and displayed

Termination

If the user enters `QUIT`, the client sends the command to the server and terminates the connection.

Error Handling

The client handles connection errors, such as when the server is not running.

Program Flow

Server Flow

1. Start server
2. Bind and listen
3. Accept client connection
4. Assign client name
5. Create a new thread
6. Receive and process commands
7. Send response
8. Log activity
9. Repeat until client disconnects

Client Flow

1. Start client
 2. Connect to server
 3. Enter command
 4. Send command
 5. Receive response
 6. Display output
 7. Repeat until QUIT
-

Error Handling

The system includes handling for:

- Invalid message formats
- Unknown commands
- Client disconnections
- Server connection failures

Here's a **professional section you can directly add to your report** with clearly defined placeholders for screenshots:

Experimental Results and Screenshots

This section presents visual evidence of the system execution, including client-server interaction, log file generation, and network packet analysis.

1. Client-Server Terminal Output

This subsection shows the interaction between the client and server through terminal windows. It includes command input from the client and corresponding responses from the server.

Description:

The screenshots below demonstrate successful communication between the client and server, including command execution and responses.

Insert Screenshot Here:

```
▼ TERMINAL

PS C:\Users\Zeus\OneDrive\Desktop\Lab2> & C:/Users/Zeus/AppData/Local/Programs/Python/Python314/python.exe c:/Users/Zeus/OneDrive/Desktop/Lab2/server.py
Server started on port 4949
```

```
PS C:\Users\Zeus\OneDrive\Desktop\Lab2> python client.py
Connected to server.

Enter command (UPPER/LOWER/REVERSE/COUNT/VOWELS/TIME/DATE/QUIT):
> Upper:Hello
Response from server: HELLO

Enter command (UPPER/LOWER/REVERSE/COUNT/VOWELS/TIME/DATE/QUIT):
> Lower:haneEN
Response from server: haneen

Enter command (UPPER/LOWER/REVERSE/COUNT/VOWELS/TIME/DATE/QUIT):
> 
```

```
PS C:\Users\Zeus\OneDrive\Desktop\Lab2> python client.py
Connected to server.

Enter command (UPPER/LOWER/REVERSE/COUNT/VOWELS/TIME/DATE/QUIT):
> Count:Makaj
Response from server: 5

Enter command (UPPER/LOWER/REVERSE/COUNT/VOWELS/TIME/DATE/QUIT):
> m
Response from server: ERROR: Unknown command from Client2.

Enter command (UPPER/LOWER/REVERSE/COUNT/VOWELS/TIME/DATE/QUIT):
> 
```

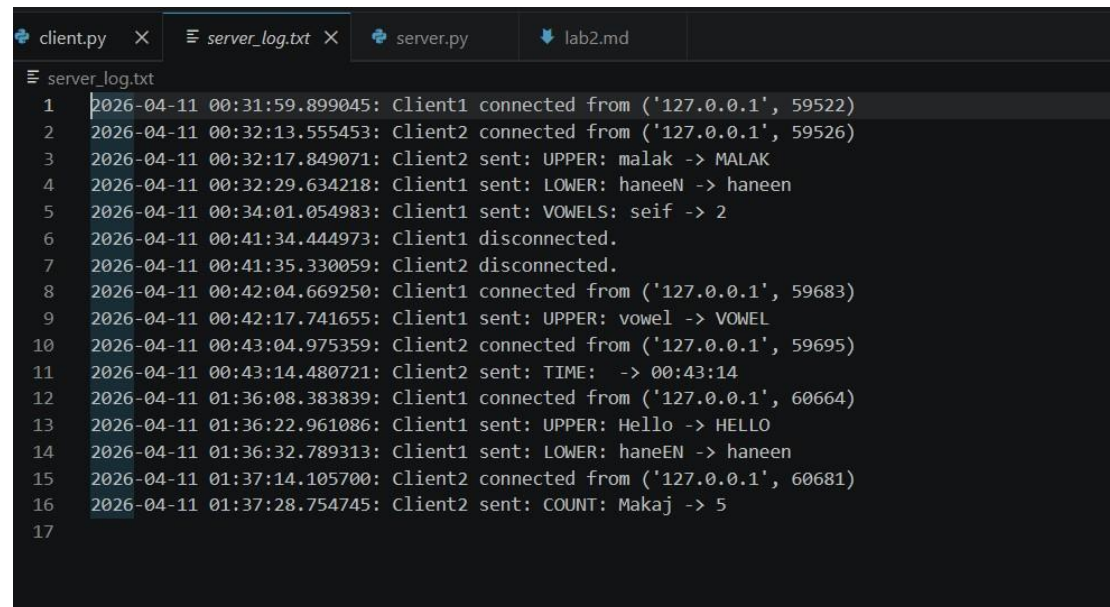
Ln 135, Col 19 Spaces: 4 UTF-8 CRLF { } Python Signed out 3.14.3 Pyt

2. Server Log File Output

This subsection presents the contents of the `server_log.txt` file, which records all client activities along with timestamps.

Description:

The log file verifies that all operations are recorded correctly, including connections, commands, and disconnections.



```
server_log.txt
1 2026-04-11 00:31:59.899045: Client1 connected from ('127.0.0.1', 59522)
2 2026-04-11 00:32:13.555453: Client2 connected from ('127.0.0.1', 59526)
3 2026-04-11 00:32:17.849071: Client2 sent: UPPER: malak -> MALAK
4 2026-04-11 00:32:29.634218: Client1 sent: LOWER: haneen -> haneen
5 2026-04-11 00:34:01.054983: Client1 sent: VOWELS: seif -> 2
6 2026-04-11 00:41:34.444973: Client1 disconnected.
7 2026-04-11 00:41:35.330059: Client2 disconnected.
8 2026-04-11 00:42:04.669250: Client1 connected from ('127.0.0.1', 59683)
9 2026-04-11 00:42:17.741655: Client1 sent: UPPER: vowel -> VOWEL
10 2026-04-11 00:43:04.975359: Client2 connected from ('127.0.0.1', 59695)
11 2026-04-11 00:43:14.480721: Client2 sent: TIME: -> 00:43:14
12 2026-04-11 01:36:08.383839: Client1 connected from ('127.0.0.1', 60664)
13 2026-04-11 01:36:22.961086: Client1 sent: UPPER: Hello -> HELLO
14 2026-04-11 01:36:32.789313: Client1 sent: LOWER: haneen -> haneen
15 2026-04-11 01:37:14.105700: Client2 connected from ('127.0.0.1', 60681)
16 2026-04-11 01:37:28.754745: Client2 sent: COUNT: Makaj -> 5
17
```

3. Network Analysis Using Wireshark

This subsection shows captured network packets using Wireshark to analyze communication between the client and server.

Description:

The screenshots illustrate TCP communication, including packet exchange, source/destination IP addresses, and transmitted data.

Insert Screenshot Here:

(Wireshark Packet Capture)

Help Tools Wireless Telephony Statistics Analyze Capture Go View Edit File									
tcp.port == 4949									
	Info	length	Protocol	Destination	Source	Time	.No		
	Seq=1 Ack=1 Win=10233 Len=2 [PSH, ACK] 4949 → 60681	46	TCP	127.0.0.1	127.0.0.1	0.000000000	1		
	Seq=1 Ack=3 Win=10233 Len=0 [ACK] 60681 → 4949	44	TCP	127.0.0.1	127.0.0.1	0.000025700	2		
	Seq=1 Ack=3 Win=10233 Len=36 [PSH, ACK] 60681 → 4949	80	TCP	127.0.0.1	127.0.0.1	0.000117300	3		
	Seq=3 Ack=37 Win=10233 Len=0 [ACK] 4949 → 60681	44	TCP	127.0.0.1	127.0.0.1	0.000134900	4		
	Seq=3 Ack=37 Win=10233 Len=5 [PSH, ACK] 4949 → 60681	49	TCP	127.0.0.1	127.0.0.1	17.806792400	5		
	Seq=37 Ack=8 Win=10233 Len=0 [ACK] 60681 → 4949	44	TCP	127.0.0.1	127.0.0.1	17.806817700	6		
	Seq=37 Ack=8 Win=10233 Len=8 [PSH, ACK] 60681 → 4949	52	TCP	127.0.0.1	127.0.0.1	17.806956300	7		
	Seq=8 Ack=45 Win=10233 Len=0 [ACK] 4949 → 60681	44	TCP	127.0.0.1	127.0.0.1	17.806974800	8		

Wireshark · Follow TCP Stream (tcp.stream eq 0) · Adapter 10

hi

ERROR: Unknown command from Client2.

TIME:

01:39:26

```
Enter command (UPPER/LOWER/REVERSE/COUNT/VOWELS/TIME/DATE/QUIT):  
> hi  
Response from server: ERROR: Unknown command from Client2.  
  
Enter command (UPPER/LOWER/REVERSE/COUNT/VOWELS/TIME/DATE/QUIT):  
> TIME:  
Response from server: 01:39:26  
  
Enter command (UPPER/LOWER/REVERSE/COUNT/VOWELS/TIME/DATE/QUIT):  
> █
```

4. Observations

- The client successfully sends commands and receives accurate responses from the server.
 - The server correctly logs all activities with timestamps.
 - Wireshark confirms that data is transmitted using TCP protocol between the client and server.
 - The system operates reliably with multiple commands and maintains proper communication flow.
-